Python Capstone Project

Student Performance and Activity Analyzer

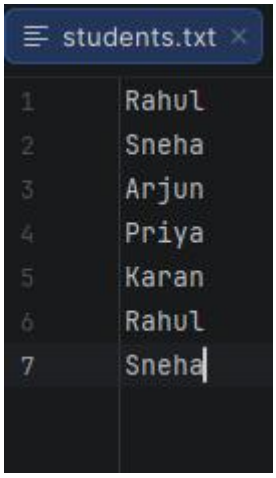
Objective

Build a Python program that reads data from:
students.txt
marks.json
attendance.csv
Then analyze the data and generate a final summary report.

Project Files

1. students.txt

Rahul
Sneha
Arjun
Priya
Karan
Rahul
Sneha



2. marks.json

{
"students": [
{"name": "Rahul", "marks": 85, "course": "Python"},
{"name": "Sneha", "marks": 92, "course": "Data Engineering"},

{"name": "Arjun", "marks": 78, "course": "Python"},
{"name": "Priya", "marks": 88, "course": "AI"},
{"name": "Karan", "marks": 70, "course": "Python"}
]

}

```
{
  "students": [
    {"name": "Rahul","marks": 85,"course": "Python"},
    {"name": "Sneha","marks": 92,"course": "Data Engineering"},
    {"name": "Arjun","marks": 78,"course": "Python"},
    {"name": "Priya","marks": 88,"course": "AI"},
    {"name": "karan","marks": 70,"course": "Python"}
  ]
}
```

3. attendance.csv

name,days_present,total_days
Rahul,22,25
Sneha,24,25
Arjun,20,25
Priya,23,25
Karan,18,25

```
name,days_present,total_days
Rahul,22,25
Sneha,24,25
Arjun,20,25
Priya,23,25
Karan,18,25
```

Part 1 — Basics and File Handling

Task 1

Read students.txt and print all names.

```
with open("students.txt","r") as f:
    data=f.readlines()
for name in data:
    print(name.strip())
```

Task 2

Count the total number of entries in students.txt .

```
print(len(data))
```

Task 3

Find the unique student names using a set.

```
unique_students=set([name.strip() for name in data])
print(unique_students)
```

Task 4

Count how many times each student name appears using a dictionary.

```
count_dict={}
for name in data:
    name=name.strip()
    count_dict[name]=count_dict.get(name, default: 0)+1
print(count_dict)
```

Task 5

Write the unique student names into a new file called unique_students.txt .

```
with open("unique_students.txt","w") as f:
    for name in unique_students:
        f.write(name+"\n")
```

Part 2 — JSON Handling

Task 6

Read marks.json .

```
import json
with open("marks.json","r") as f:
    marks_data=json.load(f)["students"]
```

Task 7

Print all student names and marks.

```
for s in marks_data:
    print(s["name"],s["marks"])
```

Task 8

Find the student with the highest marks.

```
topper=max(marks_data,key=lambda x:x["marks"])
print(topper)
```

Task 9

Find the student with the lowest marks.

```
lowest=min(marks_data,key=lambda x:x["marks"])
print(lowest)
```

Task 10

Calculate the average marks.

```
marks_list=[s["marks"] for s in marks_data]
avg=sum(marks_list)/len(marks_list)
print(avg)
```

Task 11

Print only students enrolled in the Python course.

```
for s in marks_data:
    if s["course"] == "Python":
        print(s["name"])
```

Task 12

Count how many students are there in each course using a dictionary.

```
course_count={}
for s in marks_data:
    course=s["course"]
    course_count[course]=course_count.get(course, default: 0)+1
print(course_count)
```

Part 3 — CSV Handling

Task 13

Read attendance.csv .

```
import csv
attendance={}
with open("attendance.csv","r") as f:
    reader=csv.DictReader(f)
    for row in reader:
        attendance[row["name"]]= row
```

Task 14

Print each student's attendance details.

```
for name,data in attendance.items():
    print(name,data)
```

Task 15

Calculate attendance percentage for each student.
Formula:

(days_present / total_days) * 100

```
attendance_percent={}
for name, data in attendance.items():
    percent=(int(data["days_present"]))/int(data["total_days"])*100
    attendance_percent[name]=percent
print(attendance_percent)
```

Task 16

Print students whose attendance is below 80%.

```
for name,percent in attendance_percent.items():
    if percent<80:
        print(name)
```

Task 17

Find the student with the best attendance.

```
best=max(attendance_percent,key=attendance_percent.get)
print(best)
```

Part 4 — Data Structures Practice

Task 18

Store all marks in a list and print:
highest marks
lowest marks
sum of marks

```
print(max(marks_list))
print(min(marks_list))
print(sum(marks_list))
```

Task 19

Create a tuple of all courses and print it.

```
courses_tuple=tuple([s["course"] for s in marks_data])
print(courses_tuple)
```

Task 20

Create a set of all courses to show unique courses.

```
courses_set=set(courses_tuple)
print(courses_set)
```

Task 21

Create a dictionary where:
key = student name
value = marks

```
marks_dict={s["name"]:s["marks"] for s in marks_data}
print(marks_dict)
```

Task 22

Create a second dictionary where:
key = student name
value = attendance percentage

```
attendance_dict = {}
for name, percent in attendance.items():
    attendance_dict[name] = percent
print(attendance_dict)
```

Part 5 — Conditions and Loops

Task 23

Using a loop, print whether each student is:
"Pass" if marks >= 50
"Fail" otherwise

```
for name,marks in marks_dict.items():
    if marks>=50:
        print(name,"Pass")
    else:
        print(name,"Fail")
```

Task 24

Using conditions, assign grades:
90 and above → A
75 to 89 → B
50 to 74 → C
below 50 → Fail

```
for name,marks in marks_dict.items():
    if marks>=90:
        grade="A"
    elif marks>=75:
        grade="B"
    elif marks >=50:
        grade="C"
    else:
        grade="Fail"
    print(name,grade)
```

Task 25

Print all students who have:
marks above 80
attendance above 85%

```
for name in marks_dict:
    if marks_dict[name] >80 and attendance_percent[name] >85:
        print(name)
```

Part 6 — Functions

Create separate functions for the following:

Task 26
A function to read names from students.txt .

```
def read_students(file):  1 usage
    with open(file,"r") as f:
        return[line.strip() for line in f]
students=read_students("students.txt")
print(students)
```

Task 27

A function to load student marks from marks.json .

```
def load_marks(file):  1 usage
    import json
    with open(file) as f:
        return json.load(f)["students"]
marks_data=load_marks("marks.json")
print(marks_data)
```

Task 28

A function to load attendance from attendance.csv .

```python
def load_attendance(file):  1 usage
    import csv
    result={}
    with open(file) as f:
        reader=csv.DictReader(f)
        for row in reader:
            percent=(int(row["days_present"])/int(row["total_days"]))*100
            result[row["name"]]=percent
    return result
attendance=load_attendance("attendance.csv")
print(attendance)
```

Task 29

A function to calculate average marks.

```python
def avg_marks(marks_list):  1 usage
    return sum(marks_list)/len(marks_list)
marks_list=[s["marks"] for s in marks_data]
avg=avg_marks(marks_list)
print(avg)
```

Task 30

A function to calculate attendance percentage.

```python
def attendance_percent_func(present,total):  1 usage
    return (present/total) *100
percent=attendance_percent_func( present: 22, total: 25)
print(percent)
```

Task 31

A function to return the topper.

```python
def topper_func(data):  1 usage
    return max(data,key=lambda x:x["marks"])
top=topper_func(marks_data)
print(top)
```

Task 32

A function to generate grade for a mark.

```
def grade_func(mark):  1 usage
    if mark>=90: return "A"
    elif mark>=75: return "B"
    elif mark>=50: return "C"
    else: return "Fail"
grade=grade_func(85)
print(grade)
```

Part 7 — Final Combined Analysis

Task 33

Combine marks and attendance data and create a final structure like this:

```
final_data={}
for s in marks_data:
    name=s["name"]
    final_data[name]={
        "marks":s["marks"],
        "attendance":attendance_percent[name],
        "course":s["course"]
    }
    print("Adding name",name)
print(final_data)
```

Task 34

From this combined structure, print:
name
marks
attendance
course
Grade

```
for name,data in final_data.items():
    print(name,data)
```

Task 35

Find students who are eligible for certification.
Condition:
marks >= 75
attendance >= 80

```
eligible=[]
for name,data in final_data.items():
    if data["marks"]>=75 and data["attendance"]>=80:
        eligible.append(name)
print(eligible)
```

Task 36

Find students who need improvement.
Condition:
marks < 75 or attendance < 80

```
improve=[]
for name,data in final_data.items():
    if data["marks"]<75 or data["attendance"]<80:
        improve.append(name)
print(improve)
```

Part 8 — Output File Generation

Task 37

Write the final student summary to a text file called report.txt .
{
"Rahul": {"marks": 85, "attendance": 88.0, "course": "Python"},
"Sneha": {"marks": 92, "attendance": 96.0, "course": "Data Engineerin
}

Expected style:

Student Report
Rahul - Marks: 85 - Attendance: 88.0% - Grade: B
Sneha - Marks: 92 - Attendance: 96.0% - Grade: A
Arjun - Marks: 78 - Attendance: 80.0% - Grade: B

```
with open("report.txt", "w") as f:
    f.write("Student Report\n")
    for name, data in final_data.items():
        line = f"{name} - Marks: {data['marks']} - Attendance: {round(data['attendance'],1)}% - Grade: {data['grade']}\n"
        f.write(line)
```

Task 38

Write only eligible students to eligible_students.txt .

```
with open("report.txt", "w") as f:
    for name,data in final_data.items():
        f.write(f"{name}-Marks:{data['marks']} - Attendance:{data['attendance']}\n")
```

Final Challenge

Task 39

Generate this final console output:

Topper: Sneha
Best Attendance: Sneha
Average Marks: 82.6
Eligible Students: Rahul, Sneha, Priya
Students Needing Improvement: Karan

```
print("Topper:",topper["name"])
print("Best Attendance:",best)
print("Average Marks:",round(avg,1))
print("Eligible Students:",",".join(eligible))
print("Students Needing Improvement:",",".join(improve))
```

Task 40

Make the program modular using functions and keep the code clean.

```python
def main():  1 usage
    marks_data = load_marks("marks.json")
    attendance_percent = load_attendance("attendance.csv")

    final_data = {}
    for s in marks_data:
        name = s["name"]
        final_data[name] = {
            "marks": s["marks"],
            "attendance": attendance_percent[name],
            "course": s["course"],
            "grade": generate_grade(s["marks"])
        }

    marks_list = [s["marks"] for s in marks_data]
    avg = avg_marks(marks_list)

    top = topper_func(marks_data)["name"]
    best = max(attendance_percent, key=attendance_percent.get)
```

```python
    eligible = []
    for name, data in final_data.items():
        if data["marks"] >= 75 and data["attendance"] >= 80:
            eligible.append(name)

    improve = []
    for name, data in final_data.items():
        if data["marks"] < 75 or data["attendance"] < 80:
            improve.append(name)

    with open("report.txt", "w") as f:
        f.write("Student Report\n")
        for name, data in final_data.items():
            f.write(f"{name} - Marks: {data['marks']} - Attendance: {round(data['attendance'],1)}% - Grade: {data['grade']}\n")

    print("Topper:", top)
    print("Best Attendance:", best)
    print("Average Marks:", round(avg, 1))
    print("Eligible Students:", ", ".join(eligible))
    print("Students Needing Improvement:", ", ".join(improve))

main()
```